

AI for Nuclear Thermal Hydraulics Summer School

Day 2 – Decision Trees and Ensemble models

Dr. Junwen Yin

July 14th, 2026

About me

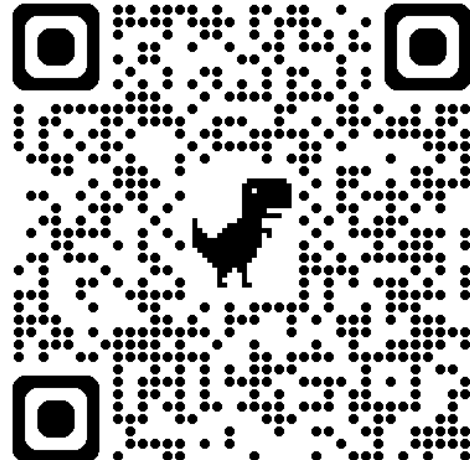
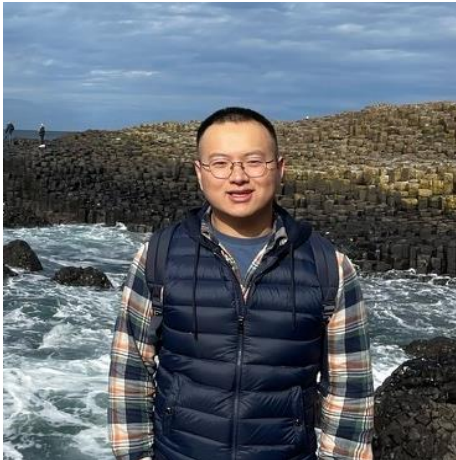
Name: Junwen Yin

Institute: Daresbury Lab, Scientific Computing Department, UKRI-STFC

Group: data-driven materials and molecular science

Theme: computational materials and molecular science

Research interests: DFT & MLIP-based multiscale materials simulations



Google Scholar

Recap: CHF datasets and pre-process

Path: CCP-NTH AI summer school/ai4nth-summer-school/data

- `chf_GP.csv`
- `chf_ong.csv`
- `chf_OOD_Kim2000.csv`
- `chf_OOD_Lee1966.csv`
- `chf_OOD_Peterlongo1966.csv`
- `chf_test.csv`
- `chf_train.csv`
- `chf_val.csv`

Questions:

- What is OOD? Why do we need it?
- What is the difference of train and validation dataset?

Recap: Linear regression

$$h(x) = w_0 + w_1x_1 + w_2x_2 + \cdots + w_nx_n$$

$$h(x) = w_0x_0 + w_1x_1 + \cdots + w_nx_n = w^T X \quad (x_0 = 1)$$

$$l(x^{(i)}) = \frac{1}{2} (h(x^{(i)}) - y^{(i)})^2$$
$$J(w) = \frac{1}{2} \sum_{i=1}^m (h(x^{(i)}) - y^{(i)})^2$$

➡ **Loss function**

Recap: Linear regression

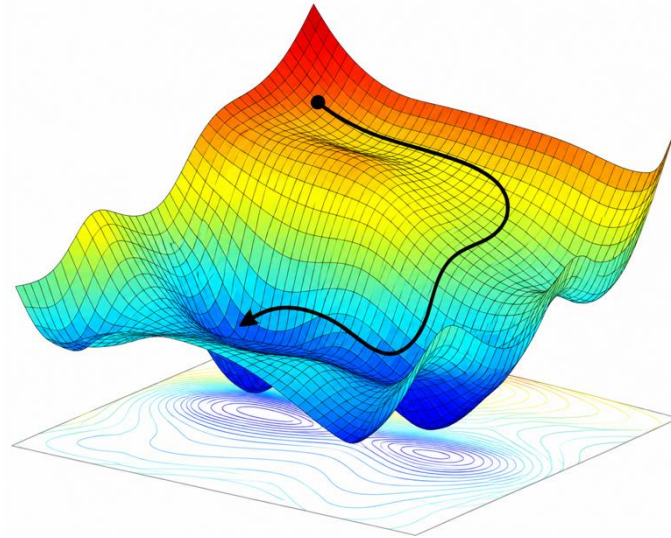
Ordinary Least Squares

$$w = (X^T X)^{-1} X^T Y$$

$$X = \begin{bmatrix} 1 & x_1^{(1)} & x_2^{(1)} & \cdots & x_n^{(1)} \\ 1 & x_1^{(2)} & x_2^{(2)} & \cdots & x_n^{(2)} \\ \vdots & \vdots & \vdots & & \vdots \\ 1 & x_1^{(m)} & x_2^{(m)} & \cdots & x_n^{(m)} \end{bmatrix}, \quad w = \begin{bmatrix} w_0 \\ w_1 \\ w_2 \\ \vdots \\ w_n \end{bmatrix}, \quad Y = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(m)} \end{bmatrix} \quad (8)$$

Gradient descent

- Batch GD
- Stochastic GD
- Mini-batch GD



Recap: L1 & L2 Regularization

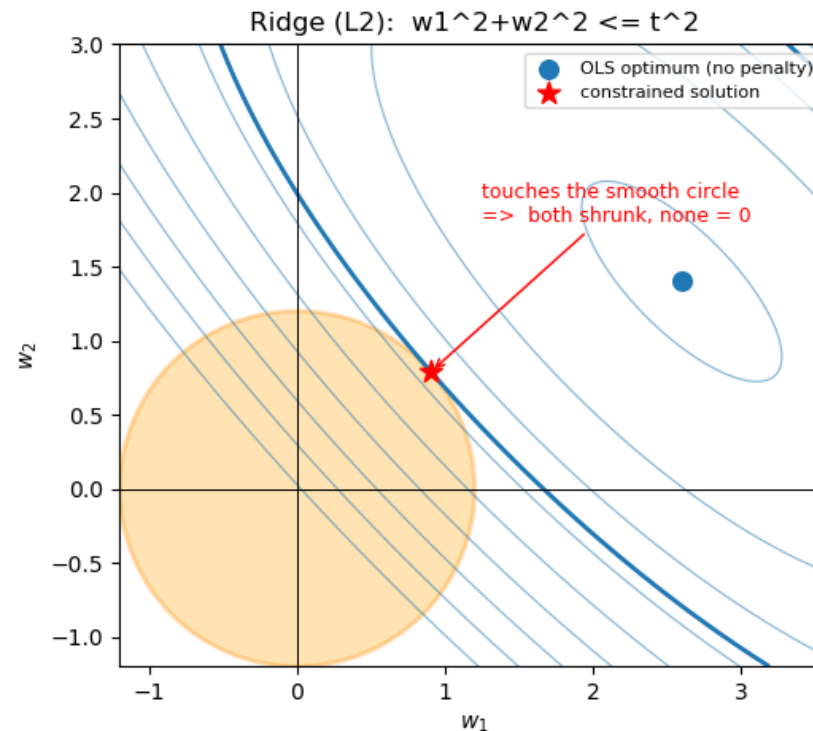
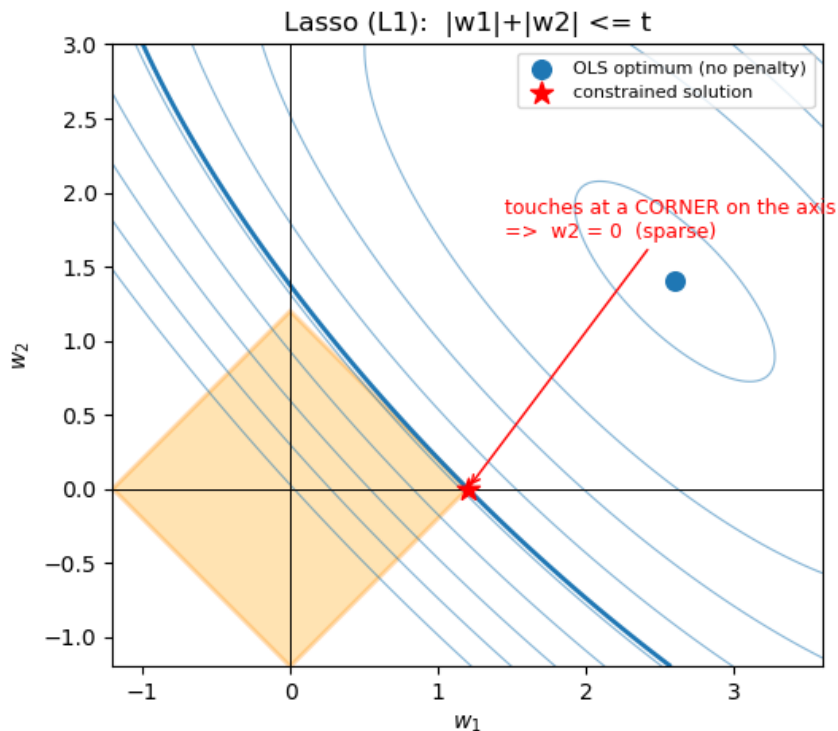
L1 Regularization (Lasso)

$$J(w) = \frac{1}{2} \sum_{i=1}^m (h(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^n |w_j|$$

L2 Regularization (Ridge)

$$J(w) = \frac{1}{2} \sum_{i=1}^m (h(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^n w_j^2$$

Why L1 -> sparse vs L2 -> smooth (geometric intuition)



Recap: Linear regression as a baseline

From linear regression to tree ensembles

WHY

- We need something beyond linear models

DAY 2 Topics

- Decision trees
- Ensemble models
 - Bagging (Random forest)
 - Boosting (Gradient Boost, Xgboost, etc.)
 - Stacking (StackingRegressor)

Decision Trees

Junwen Yin

AI4NTH Summer School - Day 2

Part 1: Decision Tree Overview

What is a decision tree?

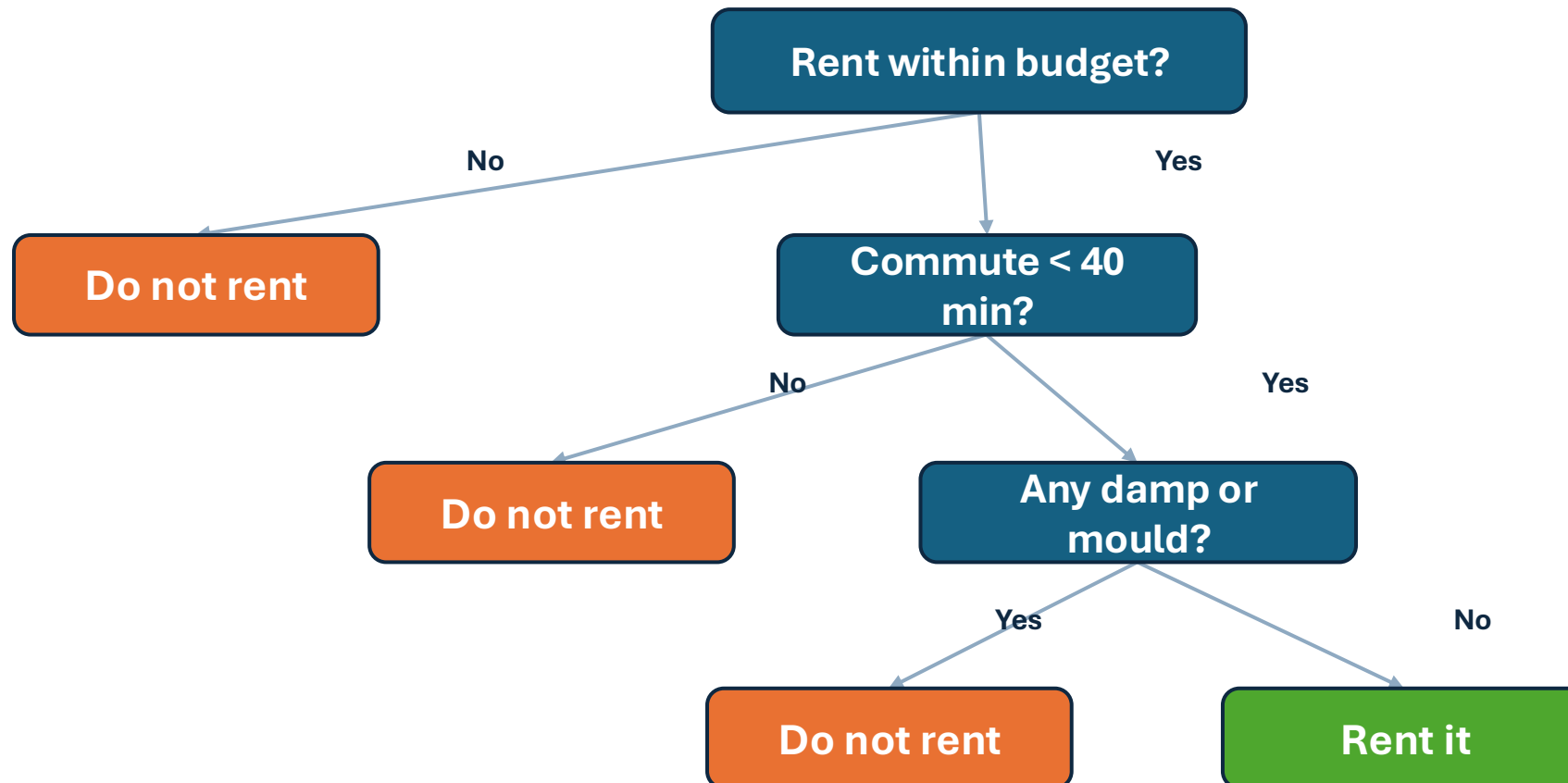
Why tree models are intuitive for supervised learning

How trees express rules through nodes, branches, and leaves

Decision Trees Start From Everyday Decisions

A decision tree formalises a familiar process: asking questions step by step.

Everyday examples: choosing a flat, deciding whether to rent, choosing between alternatives.

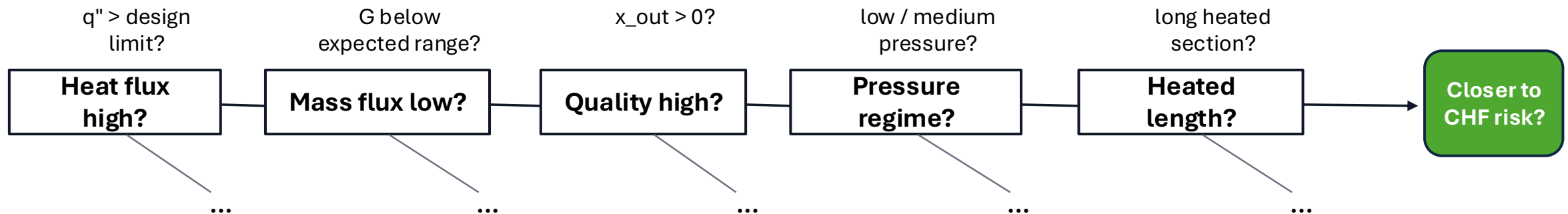


Decision Trees as Screening Rules

A decision tree is a common machine-learning model for structured decisions.

Human intuition: we often decide by checking conditions one by one.

NTH example: screening whether an operating condition is close to CHF risk.



In ML, the questions are learned from labelled data rather than written manually.

A Toy Supervised-Learning Dataset

Goal:

predict whether a condition is close to CHF risk.

Features can be discretised for intuition:

heat flux, mass flux, outlet quality, pressure.

Target:

near-CHF risk, yes or no.

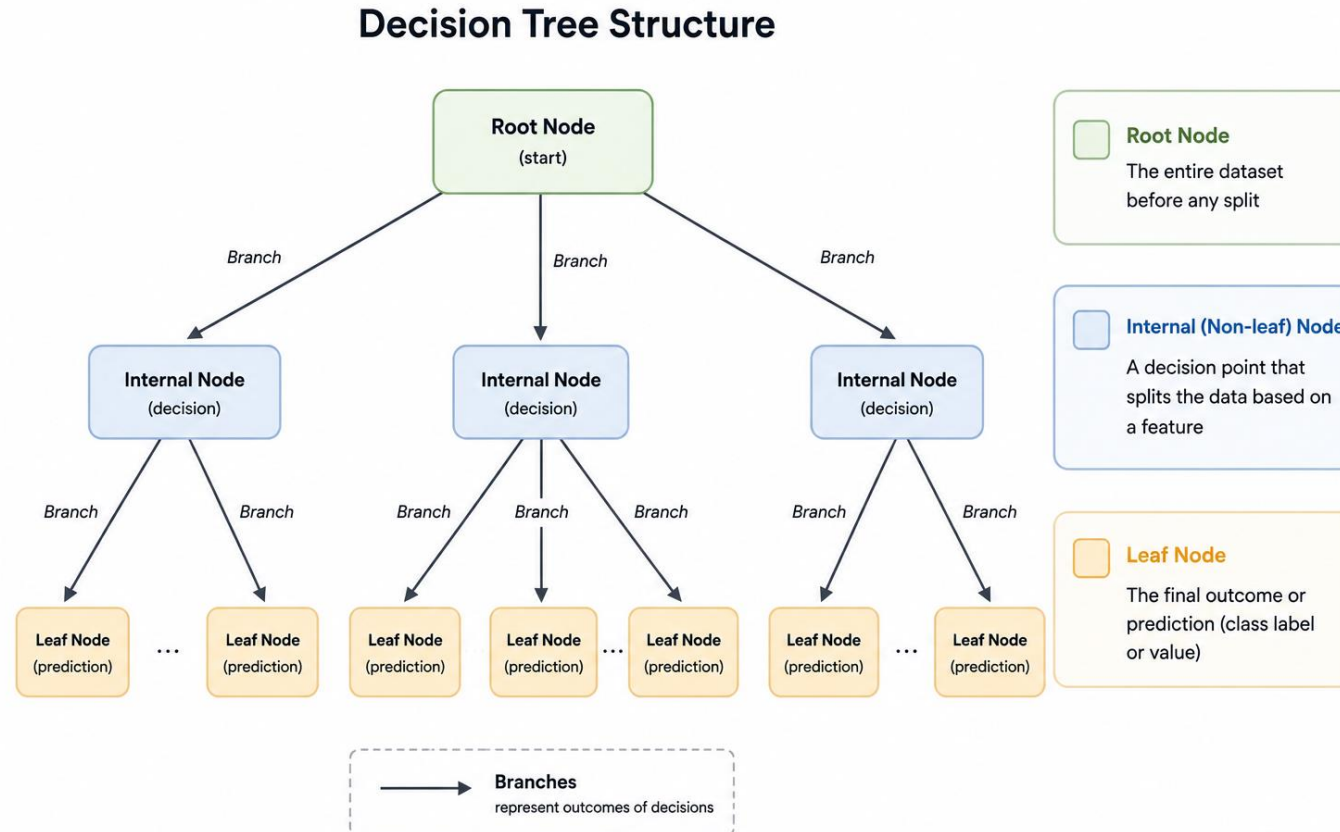
Heat flux	Mass flux	Outlet quality	Pressure	Near CHF risk?
Low	High	Low	Low	No
Low	High	Low	Low	No
Low	High	Low	Low	No
Low	Low	High	Low	No
High	High	High	Low	No
High	Low	High	High	Yes
High	High	High	High	No
High	High	Low	Low	No
High	High	High	Low	No
High	Low	Low	Low	Yes
Low	Low	High	High	Yes
High	High	High	Low	No
High	High	Low	High	Yes
Low	High	Low	High	Yes
High	Low	Low	Low	Yes

What is a Decision Tree?

A decision tree is a supervised learning method. It learns decision rules from labelled data.

Prediction starts from the root node, follows branches according to test results, and ends at a leaf.

The leaf stores the final prediction: a class label or a regression value.



Part 2: Decision Tree Algorithms

How is a tree built?

How does the algorithm choose the next split?

ID3, C4.5, and CART as three important decision-tree families

Tree-Building Workflow

Step 1: create the root node.

Step 2: for the current node, choose the best splitting attribute.

Step 3: split the data into child nodes.

Step 4: repeat recursively until a stopping condition is met.

Step 5: assign a prediction to each leaf node.

A Tree Is Built Recursively

At each node, the algorithm asks: which **split** makes the child nodes more homogeneous?

The same logic is repeated for each child node.

The process stops when the node is pure enough, too small, too deep, or no useful split remains.



How does the algorithm choose the next split?

Choosing the Best Split

The key question in tree construction is how to choose the best attribute and threshold.

Different algorithms use different split criteria.

ID3: information entropy and information gain.

C4.5: information gain ratio.

CART: Gini impurity for classification, error reduction for regression.

ID3 Algorithm

ID3 was proposed by J. Ross Quinlan in 1975.

It is an early decision-tree construction algorithm for classification.

Core idea: choose the attribute with the largest **information gain**.

Information gain measures how much uncertainty is reduced after a split.

ID3: Information Gain

A good split should make the classification result more certain.

Information gain can be measured by the change in entropy before and after splitting.

High information gain means the split reduces impurity strongly.

Entropy and Conditional Entropy

Entropy measures uncertainty in class labels.

For a dataset D , entropy is lower when one class dominates.

Conditional entropy measures remaining uncertainty after splitting on an attribute.

Information gain = parent entropy - weighted child entropy.

Entropy:

$$H(X) = - \sum_{i=1}^n p(x_i) \log p(x_i)$$

Conditional entropy:

$$H(D \mid A) = \sum_{j=1}^m \frac{|D_j|}{|D|} H(D_j)$$

A : feature, D_j : subset of D where feature A takes its j -th value

Information gain:

$$IG(D \mid A) = H(D) - H(D \mid A)$$

ID3: Approximate Procedure

1. Initialise the feature set and dataset.
2. Compute dataset entropy and conditional entropy for each feature.
3. Select the feature with the largest information gain.
4. Recursively build child nodes.
5. Stop when all samples are classified or no feature remains.

Entropy Example: Parent Dataset

Start with a labelled classification dataset.

Count how many samples belong to each class.

Compute the parent entropy from the class proportions.

15 samples in total

Near CHF risk: **6 Yes / 9 No**

Entropy: $-(6/15 * \log(6/15) + 9/15 * \log(9/15)) = 0.971$

Heat flux	Mass flux	Outlet quality	Pressure	Near CHF risk?
Low	High	Low	Low	No
Low	High	Low	Low	No
Low	High	Low	Low	No
Low	Low	High	Low	No
High	High	High	Low	No
High	Low	High	High	Yes
High	High	High	High	No
High	High	Low	Low	No
High	High	High	Low	No
High	Low	Low	Low	Yes
Low	Low	High	High	Yes
High	High	High	Low	No
High	High	Low	High	Yes
Low	High	Low	High	Yes
High	Low	Low	Low	Yes

Entropy Example: Split by One Feature

Split the dataset according to Heat Flux.

Compute the class distribution in each child subset.

A useful feature creates child subsets with lower entropy.

6 with low heat flux

Near CHF risk: **2 Yes / 4 No**

Entropy: $-(2/6 * \log(2/6) + 4/6 * \log(4/6)) = 0.918$

9 with high heat flux

Near CHF risk: **4 Yes / 5 No**

Entropy: $-(4/9 * \log(4/9) + 5/9 * \log(5/9)) = 0.991$

Split by Heat Flux

Heat flux = Low (6 samples)			
Mass flux	Outlet quality	Pressure	Near CHF risk?
High	Low	Low	No
High	Low	Low	No
High	Low	Low	No
Low	High	Low	No
Low	High	High	Yes
High	Low	High	Yes

Heat flux = High (9 samples)			
Mass flux	Outlet quality	Pressure	Near CHF risk?
High	High	Low	No
Low	High	High	Yes
High	High	High	No
High	Low	Low	No
High	High	Low	No
Low	Low	Low	Yes
High	High	Low	No
High	Low	High	Yes
Low	Low	Low	Yes

Conditional Entropy

Conditional entropy is the expected uncertainty after a split.

It combines impurity in each child node with the child-node sample size.

A split is useful when conditional entropy is much smaller than the parent entropy.

$$H(D \mid A) = \sum_{j=1}^m \frac{|D_j|}{|D|} H(D_j)$$

A : feature, D_j : subset of D where feature A takes its j -th value



$$H(D \mid \text{Heat Flux}) = \frac{6}{15} * 0.918 + \frac{9}{15} * 0.991 = 0.962$$

Information Gain

Information gain quantifies uncertainty reduction.

Information gain = entropy before split - conditional entropy after split.

The ID3 algorithm chooses the feature with the largest information gain.

Worked-example placeholder: information gain calculation for one feature.

$$H(D \mid \text{Heat Flux}) = \frac{6}{15} * 0.918 + \frac{9}{15} * 0.991 = 0.962$$

$$H(D) = 0.971$$

$$IG(D \mid \text{Heat Flux}) = 0.971 - 0.962 = 0.009$$

Comparing Information Gain Across Features

Compute information gain for every candidate feature.

Select the feature with the maximum information gain.

This selected feature becomes the split at the current node.

Feature	Information gain
Heat flux	0.009
Mass flux	0.249
Outlet quality	0.035
Pressure	0.249

Limitations of ID3

Information gain can prefer features with many possible values.

A feature with many unique values may create artificially pure child nodes.

ID3 was designed mainly for classification.

Continuous features and pruning require additional handling.

C4.5 Algorithm

C4.5 is Quinlan's improvement over ID3.

It uses information gain ratio instead of raw information gain.

Gain ratio reduces the bias toward many-valued features.

C4.5 also handles continuous attributes more naturally

Information gain: $IG(D | A) = H(D) - H(D | A)$

Split information: $IV(D | A) = - \sum_{j=1}^m \frac{|D_j|}{|D|} \log \left(\frac{|D_j|}{|D|} \right)$

Information gain ratio: $GainRatio(D | A) = \frac{IG(D | A)}{IV(D | A)}$

C4.5: Information Gain Ratio

Information gain ratio normalises information gain by split information.

This penalises splits that fragment the dataset too much.

The chosen split should be informative without creating many tiny branches.

Feature	Information gain	IV	Gain ratio
Heat flux	0.009	0.971	0.0093
Mass flux	0.249	0.918	0.2712
Outlet quality	0.035	0.997	0.0349
Pressure	0.249	0.918	0.2712

CART Algorithm

CART stands for **Classification and Regression Trees**.

CART uses binary splits.

For classification, CART commonly uses Gini impurity.

For regression, CART uses error reduction such as squared-error reduction.

$$\text{Gini impurity: } Gini(D) = \sum_{k=1}^{|y|} \sum_{k' \neq k} p_k p_{k'}$$

$$\text{Equivalent form: } Gini(D) = 1 - \sum_{k=1}^{|y|} p_k^2$$

$$\text{Gini index after splitting by feature } a: GiniIndex(D, a) = \sum_{v=1}^V \frac{|D^v|}{|D|} Gini(D^v)$$

$$\text{Gini gain: } GiniGain(D, a) = Gini(D) - GiniIndex(D, a)$$

CART: Gini Index

Gini impurity measures how mixed the classes are in a node.

A pure node has $\text{Gini} = 0$.

A good split reduces weighted Gini impurity in the child nodes.

Feature	Gini index
Heat flux	0.006
Mass flux	0.160
Outlet quality	0.023
Pressure	0.160

Part 3: More Details About CART

How CART handles different feature types

How decision trees support regression

How decision boundaries arise

CART in scikit-learn

scikit-learn decision trees are based on CART-style algorithms.

Feature selection: choose the split that optimises the selected criterion.

Classification and regression use different split criteria.

Practical training also depends on stopping and pruning parameters.

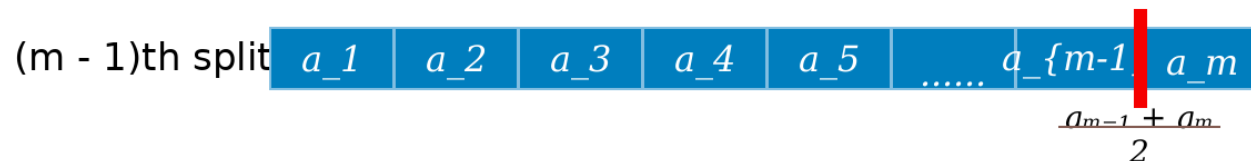
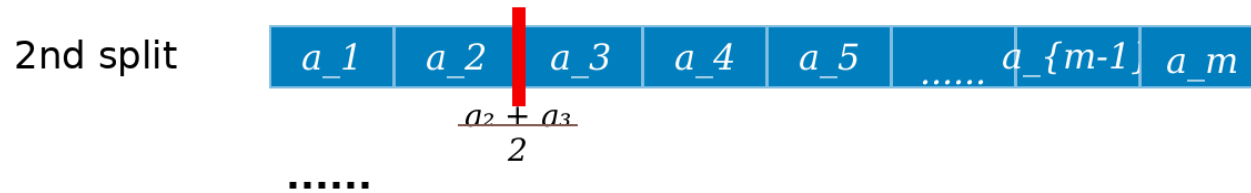
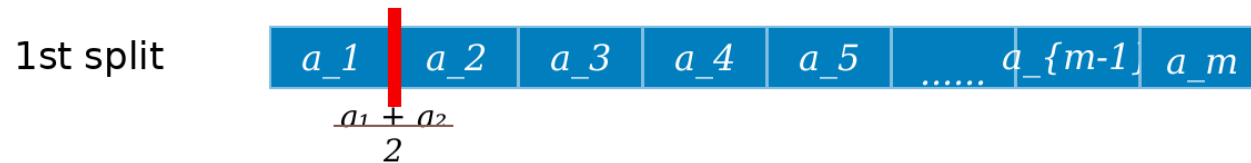
CART Classification: Continuous Features

Continuous features must be converted into threshold tests.

For sorted feature values, candidate thresholds are placed between adjacent values.

The algorithm evaluates each threshold and chooses the best split.

Example split: feature $\leq$ threshold versus feature $>$ threshold.



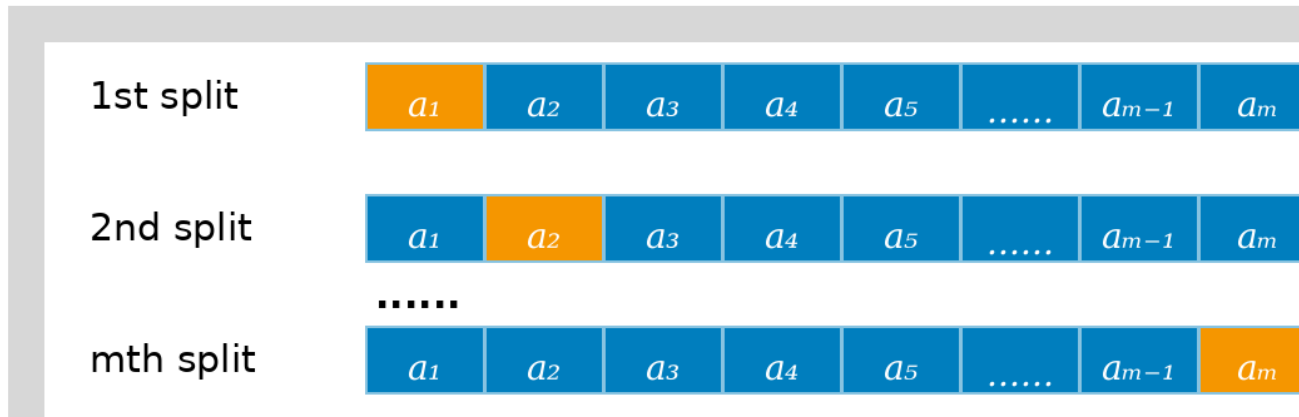
CART Classification: Multi-Category Features

For categorical features with multiple values, CART needs a binary partition.

One strategy is to split one category versus the remaining categories.

Different partitions can be compared by impurity reduction.

Categorical handling depends on the implementation and preprocessing.



For example, in the 1st split, $\{a_1\}$ is assigned to class 1, and the remaining values $\{a_2, a_3, \dots, a_{m-1}, a_m\}$ are assigned to class 2. The process is repeated until the m-th split.

CART and Categorical Features in sklearn

scikit-learn generally expects numerical input.

Multi-category variables are usually preprocessed before modelling.

Common preprocessing: one-hot encoding.

After encoding, the tree splits on binary indicator features.

CART for Regression

The previous impurity criteria were mainly for classification.

For regression, leaf predictions are numerical values.

A regression tree chooses splits that reduce prediction error.

Typical leaf prediction: the mean target value of training samples in that leaf.

Regression Trees: Change the Split Objective

Classification split objective: reduce class impurity.

Regression split objective: reduce numerical prediction error.

Instead of entropy or Gini, use criteria such as MSE or MAE.

The tree structure is similar; the meaning of leaf output changes

Regression split criteria:

$$MSE = \frac{1}{M} \sum_{i=1}^M (y_i - \hat{y})^2$$

$$MAE = \frac{1}{M} \sum_{i=1}^M |y_i - \hat{y}|$$

Regression Split Criteria

MSE criterion: choose splits that reduce mean squared error.

MAE criterion: choose splits that reduce absolute error.

MSE-based leaves often predict the mean.

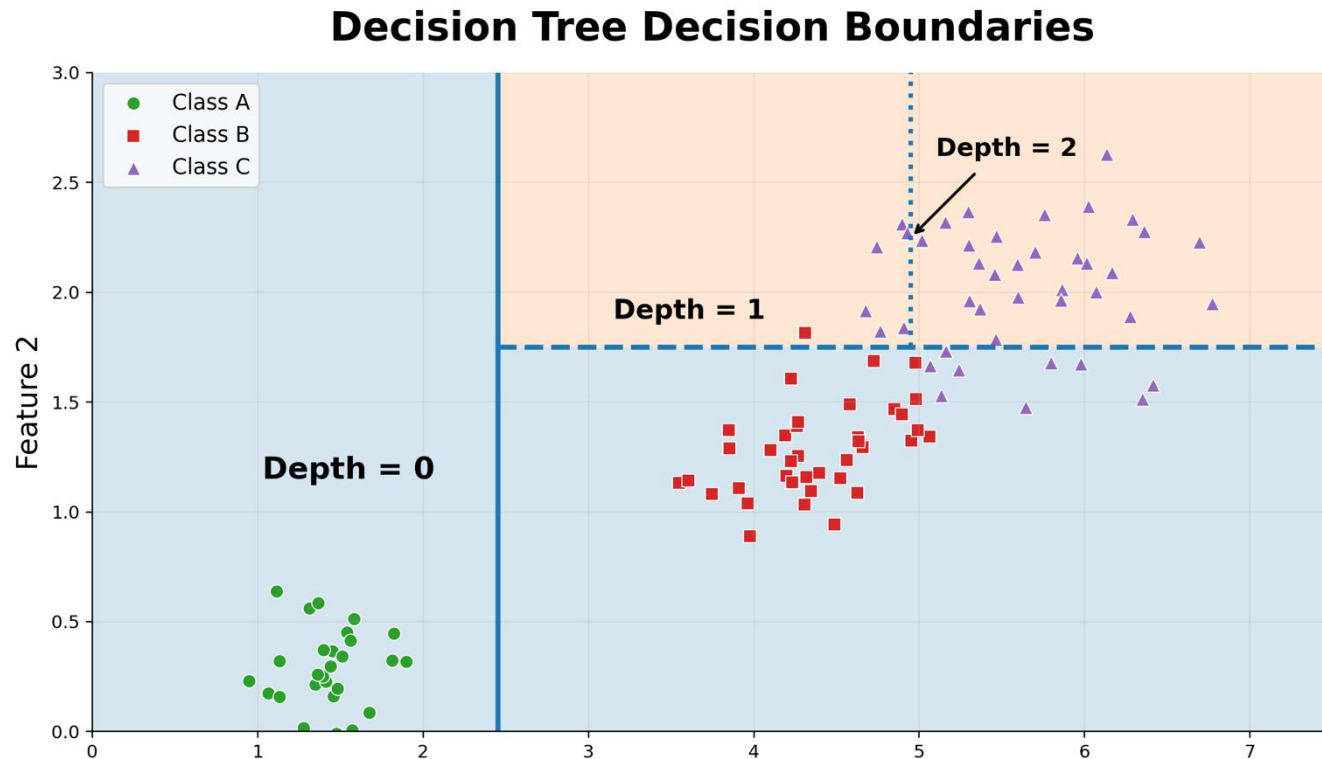
MAE-based leaves are related to median-style predictions.

Decision Boundary

A decision tree partitions feature space into regions.

For classification, boundaries separate predicted classes.

For regression, regions have piecewise-constant predictions.



Part 4: Overfitting and Pruning

Why trees overfit

Pre-pruning and post-pruning

How sklearn controls tree complexity

Overfitting

A deep tree can keep splitting until it fits training samples almost perfectly.

This can make the model learn noise or accidental patterns.

Overfitting gives low training error but poor generalisation.

Tree complexity must therefore be controlled.

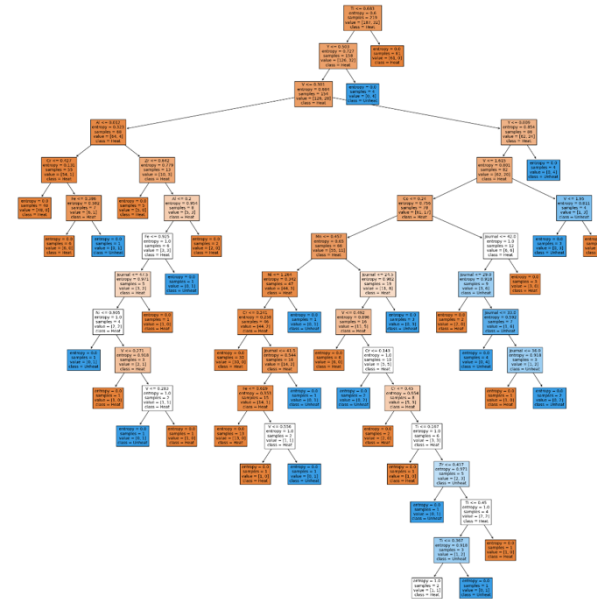
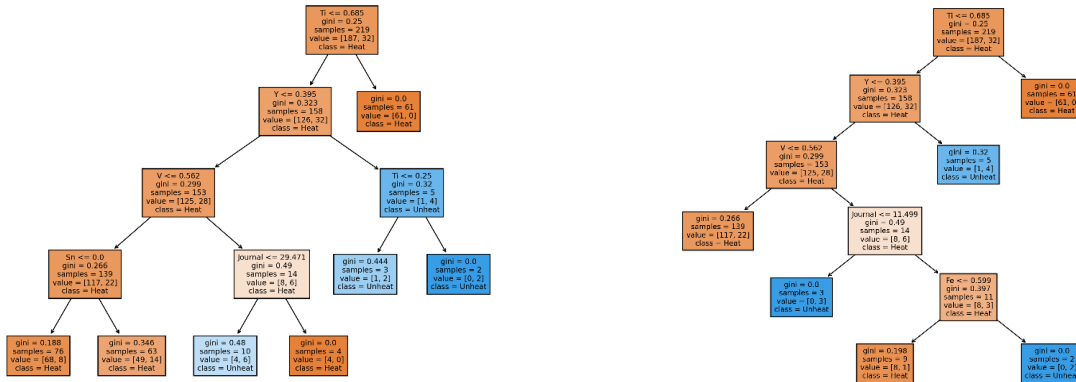
Pruning in sklearn

In sklearn, tree complexity is often controlled by limiting depth and leaf sizes.

Important controls: `max_depth`, `min_samples_split`, `min_samples_leaf`, `max_leaf_nodes`.

These settings act as pre-pruning or growth constraints.

Cost-complexity pruning provides a post-pruning route.



Pre-Pruning

Pre-pruning stops tree growth early.

It can reduce overfitting and training time.

Because it is greedy, it may stop a split that would have been useful later.

Use validation performance to tune pre-pruning parameters.

Pre-Pruning Strategies

Stop splitting when a node has too few samples.

Stop when all features have already been used or no useful split remains.

Stop when a split does not improve validation performance enough.

Limit maximum tree depth or maximum leaf count.

Pre-Pruning: Practical Controls

`max_depth`: maximum depth of the tree.

`min_samples_leaf`: minimum samples required in a leaf.

`min_samples_split`: minimum samples required before splitting.

`max_leaf_nodes`: maximum number of leaves.

Post-Pruning

Post-pruning first grows a larger tree, then simplifies it.

It removes branches that do not improve validation performance enough.

Post-pruned trees often keep more useful structure than aggressively pre-pruned trees.

Validation data are used to decide how much pruning is appropriate.

Entropy-Based Post-Pruning: Optional Reading

A post-pruning method can work bottom-up through the tree.

At each internal node, compare keeping the subtree versus replacing it with a leaf.

If pruning does not hurt validation performance, simplify the subtree.

This illustrates the general idea of reducing unnecessary complexity.

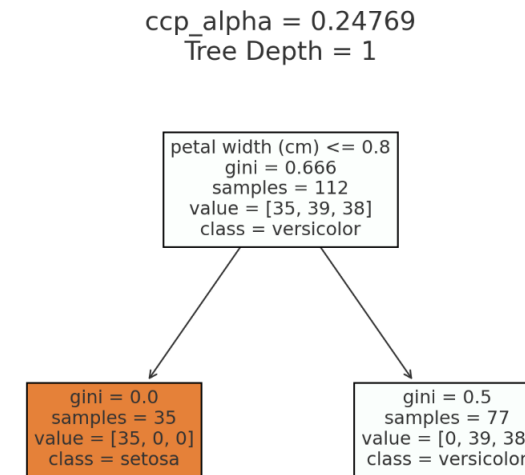
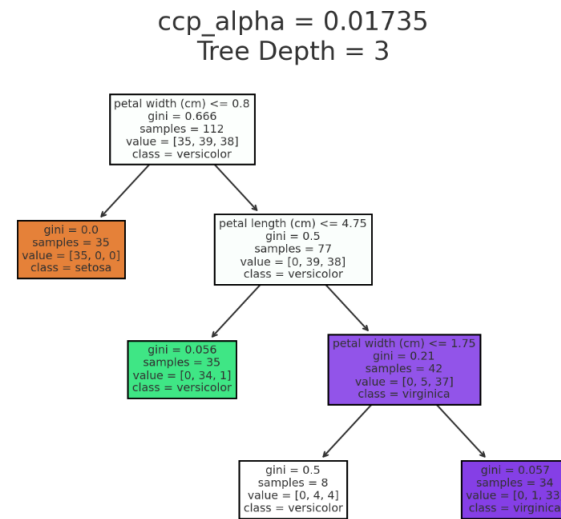
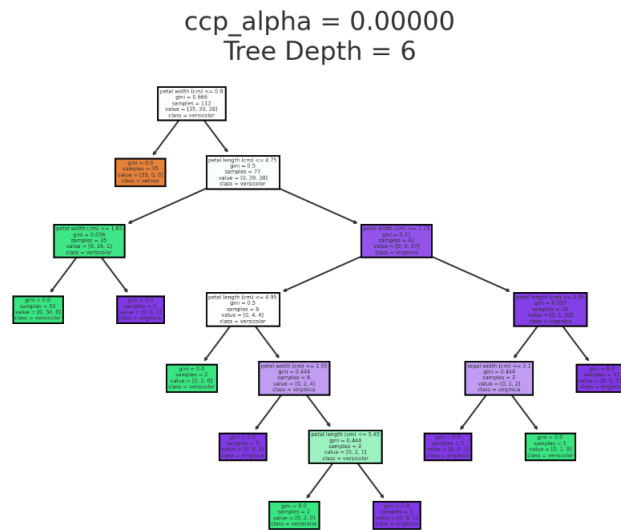
Post-Pruning in sklearn

sklearn supports cost-complexity pruning.

The cost-complexity objective adds a penalty for tree size.

ccp_alpha controls the strength of the complexity penalty.

Larger ccp_alpha gives a smaller, more pruned tree.



Part 5: Implementation in sklearn

Decision-tree classes in sklearn

Basic modelling workflow

Important parameters and attributes

Decision Trees in sklearn

Decision-tree models live in `sklearn.tree`.

Key classes include `DecisionTreeClassifier` and `DecisionTreeRegressor`.

Related tools include `export_text`, `plot_tree`, and `export_graphviz`.

Use classifier for labels and regressor for continuous targets.

scikit-learn API	Description
<code>tree.DecisionTreeClassifier</code>	Decision tree classifier
<code>tree.DecisionTreeRegressor</code>	Decision tree regressor
<code>tree.export_graphviz</code>	Export a trained tree in DOT format for visualization
<code>tree.ExtraTreeClassifier</code>	Extremely randomized tree classifier
<code>tree.ExtraTreeRegressor</code>	Extremely randomized tree regressor

Basic sklearn Modelling Workflow

1. Import the model class.
2. Create a model object with chosen parameters.
3. Fit the model on training data.
4. Predict on validation or test data.
5. Evaluate performance with suitable metrics.

DecisionTreeClassifier Interface

DecisionTreeClassifier is used for classification tasks.

Important inputs include criterion, splitter, max_depth, min_samples_split, and min_samples_leaf

The fitted tree stores learned structure and thresholds.

Use predict or predict_proba depending on the task.

- Classification tree in scikit-learn

```
class sklearn.tree.DecisionTreeClassifier(criterion='gini', splitter='best',
max_depth=None, min_samples_split=2, min_samples_leaf=1,
min_weight_fraction_leaf=0.0, max_features=None, random_state=None,
max_leaf_nodes=None, min_impurity_decrease=0.0, class_weight=None)
```

- Regression tree in scikit-learn

```
class sklearn.tree.DecisionTreeRegressor(criterion='squared_error', splitter='best',
max_depth=None, min_samples_split=2, min_samples_leaf=1,
min_weight_fraction_leaf=0.0, max_features=None, random_state=None,
max_leaf_nodes=None, min_impurity_decrease=0.0)
```

Parameter Explanation: criterion

criterion defines how split quality is measured.

For classification, common criteria include gini, entropy, and log_loss.

The criterion helps the algorithm find the best node split.

Changing criterion can change the learned tree.

criterion in More Detail

criterion measures impurity or prediction error.

Classification criteria: Gini impurity, entropy, log loss.

Regression criteria: squared_error, friedman_mse, absolute_error, poisson.

Choose the criterion based on task type and modelling goal.

Important Fitted Attributes

After fitting, sklearn trees expose useful attributes.

Examples: `feature_importances_`, `n_features_in_`, `classes_`, `tree_`.

These attributes help inspect what the model learned.

Code placeholder: simple `DecisionTreeClassifier` example.

Decision Tree Workflow Summary

The tree repeatedly chooses the best available split.

It stops when there are no useful splits or stopping conditions are met.

Leaves store the predicted class or numerical value.

Model complexity controls are essential for generalisation.

Ensemble Learning Algorithms and Materials Discovery

Junwen Yin

AI4NTH Summer School - Day 2

Course Outline

Ensemble learning overview

Main families of ensemble learning

Five common ensemble algorithms

Paper-reproduction / application case

Summary of classical machine-learning algorithms

Ensemble Learning Overview

So far, students may have seen linear regression, logistic regression, decision trees, and SVMs.

Those models improve prediction by strengthening a single model form.

Ensemble learning takes a different route: combine multiple models.

Each individual model can be relatively weak, but the combined model can be strong.

Key phrase: strength through aggregation.

Main Ensemble Type: Boosting

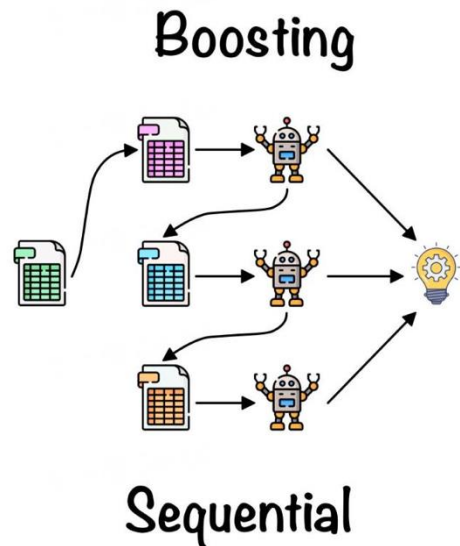
Boosting trains a sequence of weak learners.

Each new learner tries to compensate for weaknesses of previous learners.

The weakness can be poorly fitted samples or remaining prediction errors.

Training is sequential: later models depend on earlier models.

Representative algorithms: AdaBoost, Gradient Boosting, XGBoost, LightGBM.



Main Ensemble Type: Bagging

Bagging means Bootstrap Aggregating.

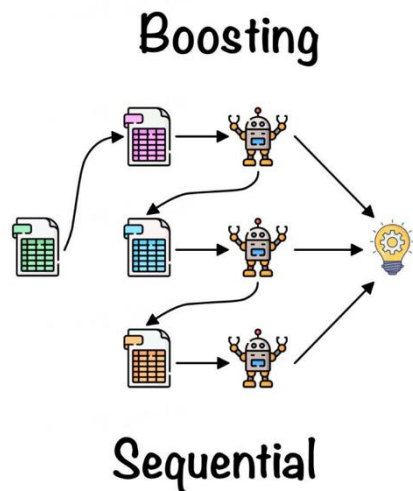
Core sampling idea: draw bootstrap samples with replacement.

Train one base learner on each sampled dataset.

For classification, combine models by voting.

For regression, combine models by averaging.

Representative algorithm: Random Forest



Main Ensemble Type: Stacking

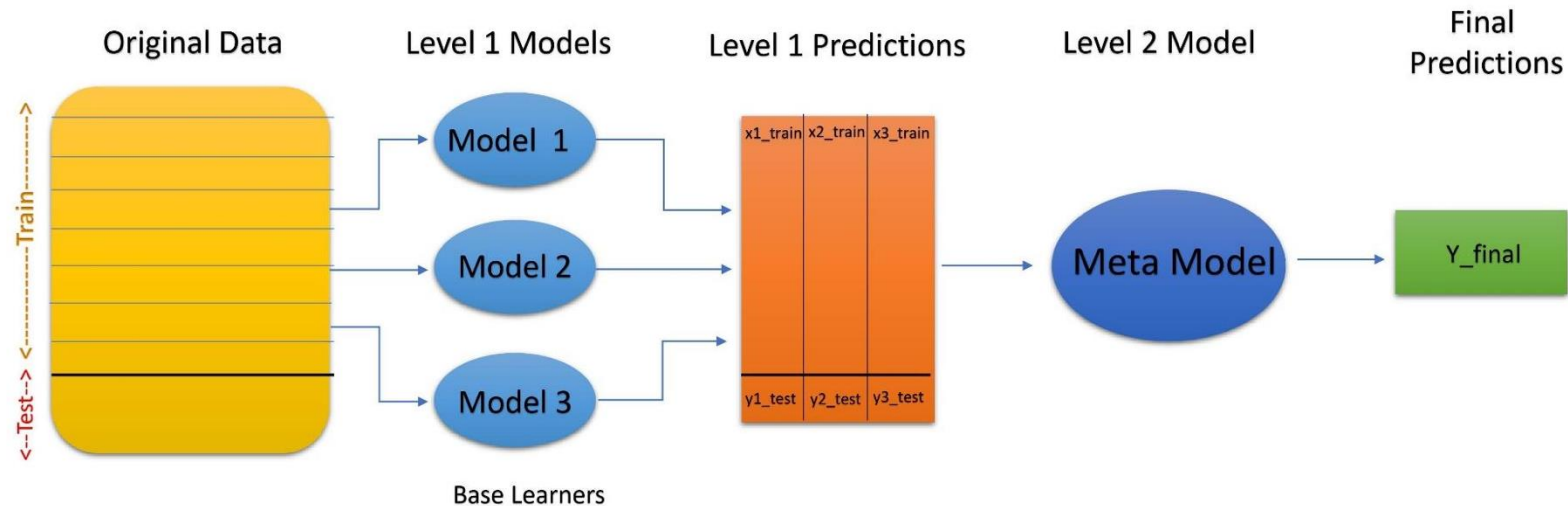
Stacking uses multiple learners in layers.

First-level models generate predictions.

A second-level model learns how to combine those predictions.

The second-level learner uses first-level outputs as input features.

Stacking is powerful but needs careful validation to avoid leakage



Common Ensemble Algorithm: Random Forest

Random Forest is a common bagging algorithm.

Base learners are decision trees.

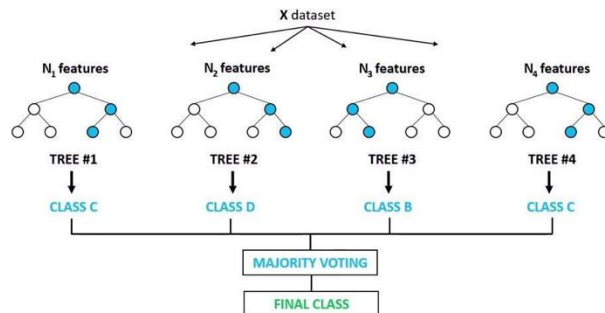
Each tree is trained with two kinds of randomness:

Data randomness: bootstrap sampling.

Feature randomness: each split considers only a random subset of features.

Advantages: simple, efficient, robust, and less prone to overfitting than one deep tree

Random Forest Classifier



Random Forest in sklearn

Classification: RandomForestClassifier.

Regression: RandomForestRegressor.

Important parameters:

n_estimators: number of trees.

max_features: number of candidate features per split.

max_depth: maximum tree depth.

bootstrap: whether bootstrap sampling is used.

n_jobs: number of CPU cores.

Common Ensemble Algorithm: AdaBoost

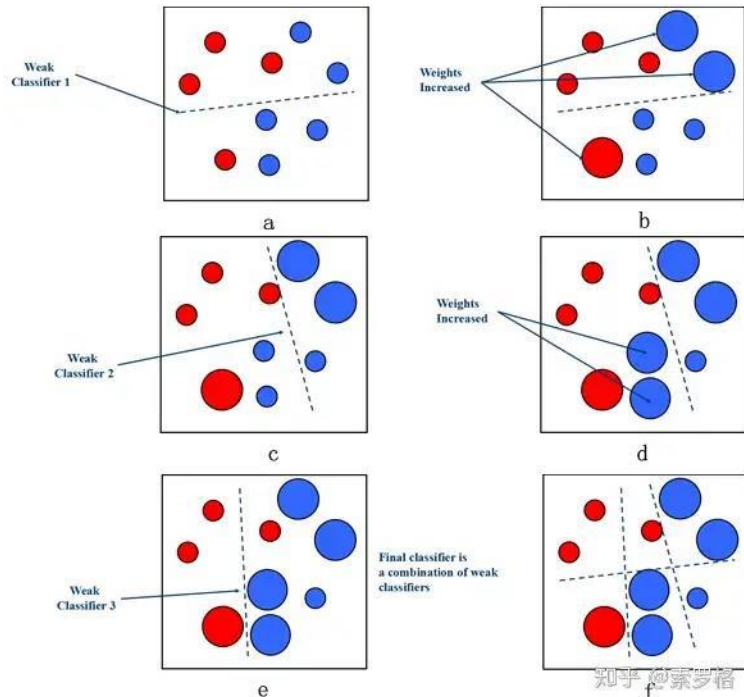
AdaBoost is a classic boosting algorithm.

Core idea: focus more on samples that previous learners handled poorly.

Misclassified or poorly predicted samples receive higher weight.

Each learner is trained on a reweighted version of the data.

Final prediction combines learners with different weights



AdaBoost in sklearn

Classification: AdaBoostClassifier.

Regression: AdaBoostRegressor.

Common parameters:

estimator: the base learner.

n_estimators: number of boosting rounds.

learning_rate: contribution of each learner.

loss: regression loss function for AdaBoostRegressor

AdaBoost classifier with the default base learner

```
# Create an AdaBoost classifier
ada_clf = AdaBoostClassifier(
    n_estimators=50,
    learning_rate=1.0,
    random_state=42
)

# Train the model
ada_clf.fit(X_train, y_train)

# Make predictions
y_pred = ada_clf.predict(X_test)
```

AdaBoost classifier with Logistic Regression

```
# Create Logistic Regression as the base learner
log_reg = LogisticRegression(max_iter=1000)

# Create an AdaBoost classifier with Logistic Regression
ada_clf_with_log_reg = AdaBoostClassifier(
    estimator=log_reg,
    n_estimators=50,
    learning_rate=1.0,
    random_state=42
)

# Train the model
ada_clf_with_log_reg.fit(X_train, y_train)

# Make predictions
y_pred = ada_clf_with_log_reg.predict(X_test)
```

Common Ensemble Algorithm: Gradient Boosting

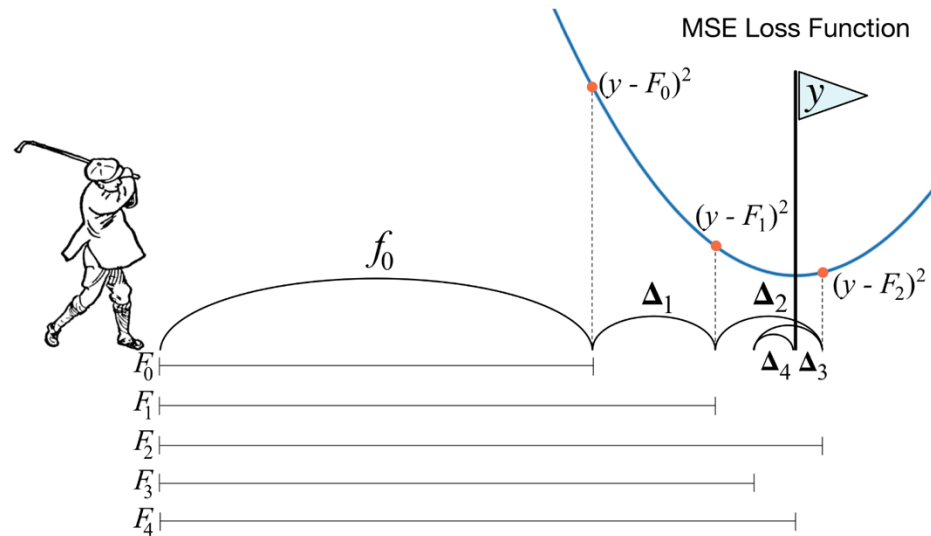
Gradient Boosting is another major boosting method.

Core idea for regression: fit new learners to the residual errors.

The model is additive: each tree adds a correction.

It can reduce bias and achieve high accuracy.

It is more tuning-sensitive than Random Forest.



Gradient Boosting in sklearn

Classification: GradientBoostingClassifier.

Regression: GradientBoostingRegressor.

Modern sklearn option: HistGradientBoostingRegressor / Classifier.

Important parameters:

n_estimators or max_iter: number of boosting steps.

learning_rate: size of each correction.

max_depth or max_leaf_nodes: complexity of each tree.

subsample: stochastic gradient boosting if less than 1.

Common Ensemble Algorithm: XGBoost

XGBoost stands for eXtreme Gradient Boosting.

It is an efficient implementation of gradient boosting.

It adds strong regularisation and systems-level optimisation.

Often performs very well on tabular data.

Commonly used in machine-learning competitions and applied materials/engineering studies



Common Ensemble Algorithm: LightGBM

LightGBM is another efficient gradient boosting framework.

It is designed for speed and scalability.

It often works well for large tabular datasets.

It uses histogram-based split finding and leaf-wise tree growth.

Like XGBoost, it is separate from core scikit-learn



LightGBM

Using LightGBM

LightGBM provides classifier and regressor interfaces.

Typical use: install package, prepare tabular data, define model, fit, evaluate.

Important parameters:

`num_leaves`: controls tree complexity.

`learning_rate` and `n_estimators`: boosting schedule.

`feature_fraction` and `bagging_fraction`: stochastic training.

regularisation parameters: reduce overfitting.

LightGBM: Practical Notes

LightGBM can be very fast, but parameter tuning matters.

Leaf-wise growth can improve accuracy but may overfit if unconstrained.

Control model complexity with `num_leaves`, `min_data_in_leaf`, and `max_depth`.

Use validation data and early stopping where possible.

Figure/code placeholder: LightGBM training workflow.

Paper-Reproduction / Application Case

The source PDF ends with a paper-reproduction case section.

For our course, this can become a materials or NTH model-comparison case.

Suggested Day 2 adaptation:

compare Ridge, Decision Tree, Random Forest, and Gradient Boosting.

evaluate overall metrics and regime-wise diagnostics.

interpret feature importance cautiously.

Part 6: Machine-Learning Interpretability

Why interpretability matters

Intrinsic versus post-hoc interpretability

How decision trees connect prediction and explanation

Two Types of Interpretability

Intrinsic interpretability: the model structure is directly understandable.

Post-hoc interpretability: analyse a trained model after fitting.

Decision trees can be intrinsically interpretable when small.

Complex ensembles usually require post-hoc tools.

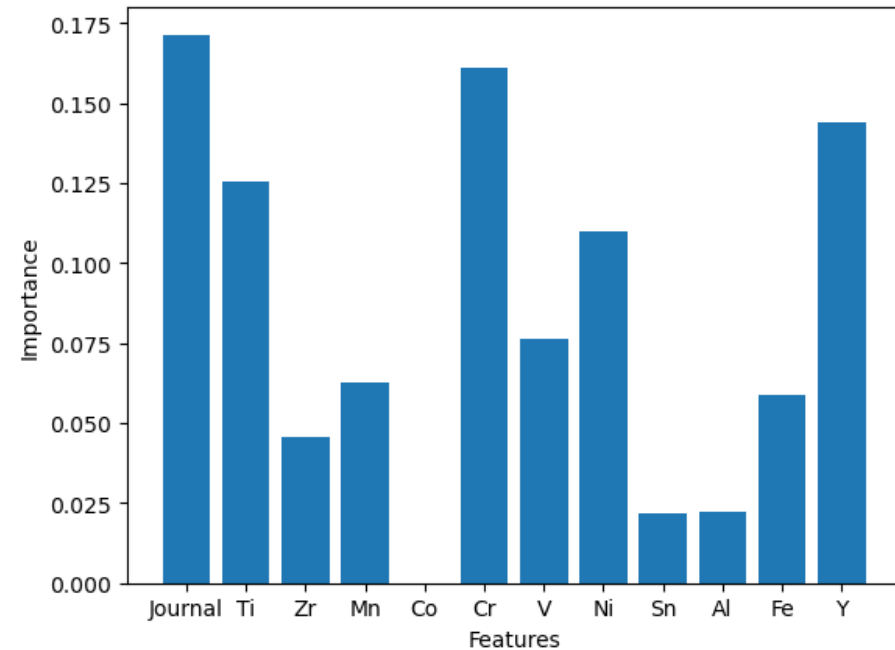
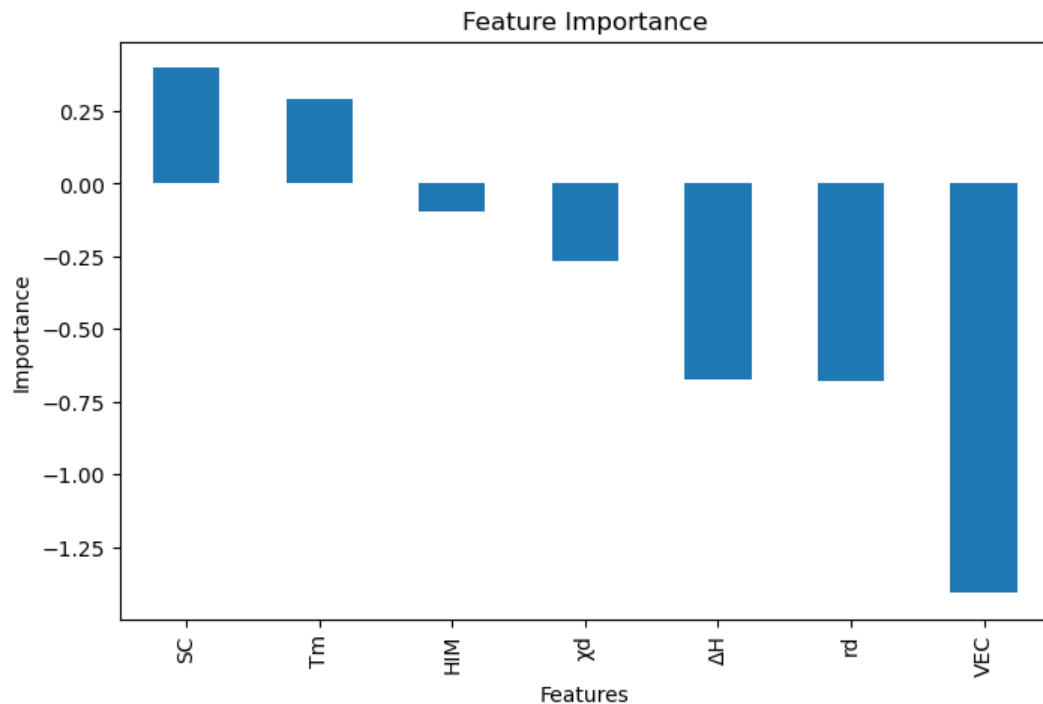
Intrinsic Interpretability

The model itself is transparent enough to inspect.

Examples: small decision trees and linear models.

We can trace a prediction path from root to leaf.

Caution: a large tree may no longer be easy to understand.

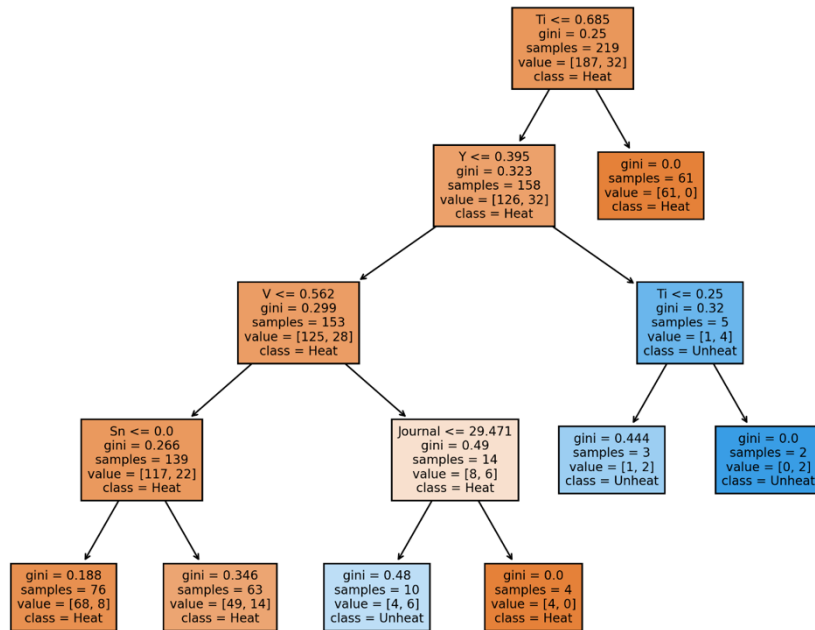


Intrinsic Interpretability: Example

A single shallow tree can show which features are used first.

The path gives a local explanation for one prediction.

The tree structure can suggest data regimes.



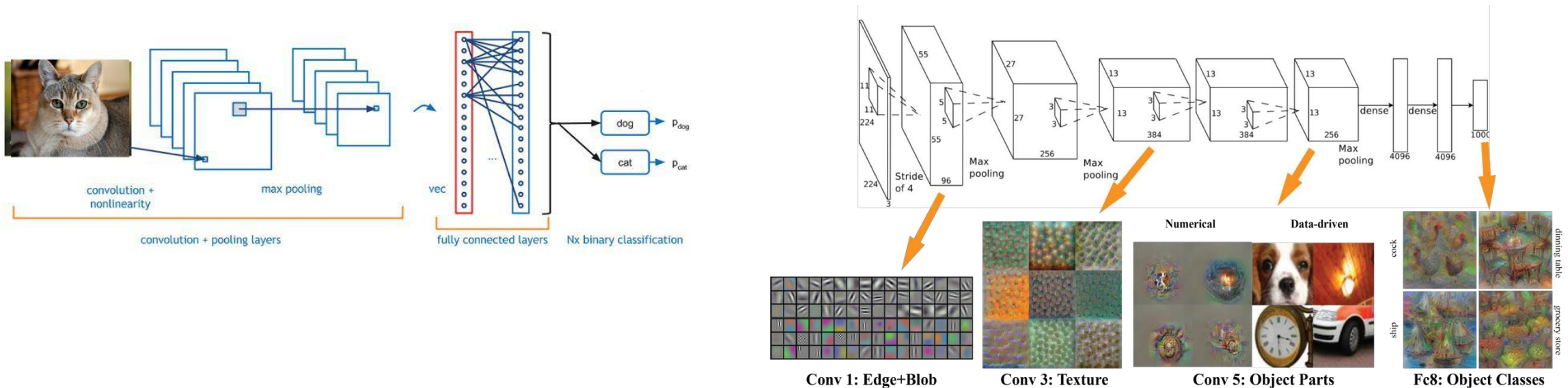
Post-Hoc Interpretability

Post-hoc methods explain a model after training.

They are useful for complex models such as ensembles or neural networks.

Examples: feature importance, permutation importance, partial dependence, SHAP.

These methods explain model behaviour, not necessarily physical causality



Is Interpretability Really Important?

Discussion question: when do we need to understand a model?

In scientific and engineering settings, prediction accuracy is not always enough.

Reliability, failure modes, and domain consistency also matter.

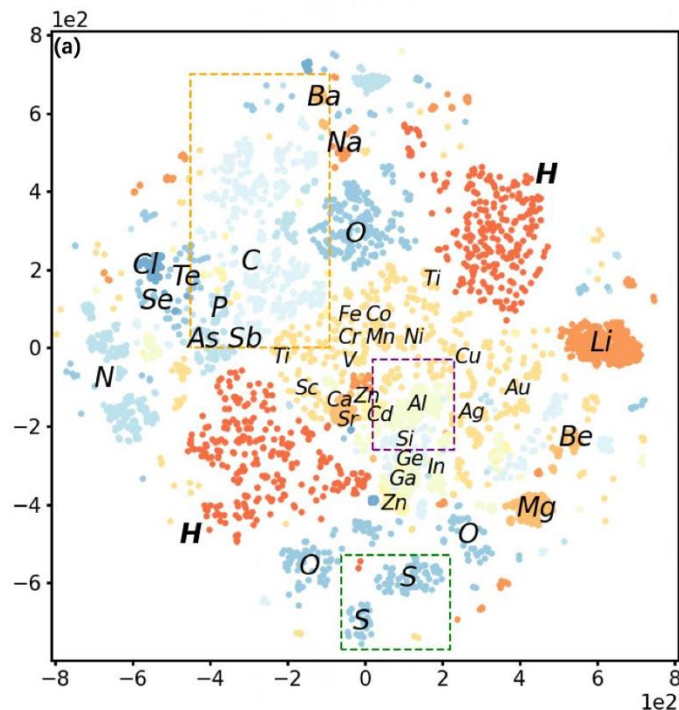
Why Interpretability Is Important

Model stability: some domains cannot accept unstable outputs.

Model correctness: explanations help identify unreasonable behaviour.

Trust and communication: users need to understand why a model makes a prediction.

Scientific insight: interpretation can guide further investigation



When Interpretability May Be Less Central

Some high-performing systems are useful even if they are partly black-box.

Human reasoning itself is not fully transparent.

If prediction is the only goal and risk is low, performance may dominate.

For NTH, however, interpretability remains important because decisions are scientific and safety-relevant.

Part 7: ChatGPT and Large Models

A short prompt-engineering example

Using large models to generate teaching or analysis cases

Caution: always verify technical outputs

Prompting Large Models for Teaching Examples

A good prompt should describe the dataset, features, target, and teaching goal.

Example: ask for a decision-tree teaching case with high/low features and a yes/no target.

Large models can help draft examples, but the instructor must verify correctness.

Thanks

THANKS

Questions?

Next: connect ensemble concepts to the Day 2 notebook and hands-on CHF modelling.

Using XGBoost

XGBoost is not part of core scikit-learn.

It is installed as a separate package.

It provides sklearn-like interfaces such as `XGBClassifier` and `XGBRegressor`.

Common workflow: install package, define model, fit, predict, evaluate.

Important: tune learning rate, number of trees, tree depth, and regularisation.

XGBoost: Model-Control Ideas

XGBoost includes many controls for model complexity and training behaviour.

Typical controls include:

tree depth and number of leaves.

learning rate and number of boosting rounds.

row and feature subsampling.

L1/L2 regularisation.

Figure/code placeholder: XGBoost parameter categories.

XGBoost: Loss Function Configuration

XGBoost supports different objective functions.

The objective should match the task:

regression, classification, ranking, or count modelling.

For engineering regression, squared-error or related objectives are common.

Documentation placeholder: objective/loss-function table.